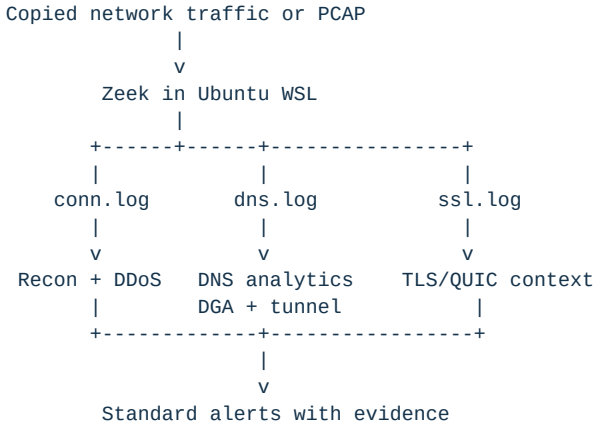
AEGISFLOW

Passive AI-Assisted Cyber-Threat Detection

REPORT	SIH Engineering Build Walkthrough
COVERAGE	Sprints 0 to 3
STATUS	35 automated tests passing
SNAPSHOT	29 August 2026

1. What we are building

AegisFlow observes copied network traffic on the monitoring side of a one-way or passively monitored network. It does not send packets back toward the protected network. Zeek converts packet captures into structured connection, DNS, and encrypted-session metadata. AegisFlow then applies transparent statistical detectors and focused machine-learning models to produce evidence-rich alerts.



2. Current progress at a glance

Sprint	Main outcome	Status	Demonstration result
Sprint 0	Shared contracts, Zeek connection ingestion, scan detection	Complete	Vertical and horizontal scan alerts
Sprint 1	WSL Zeek bridge, real PCAP processing, SYN/UDP flood detection	Complete	Real PCAP: 12 connections processed
Sprint 2	DNS ingestion, DGA model, DNS-tunnelling detector	Prototype complete	Threat fixture: 21 events, 2 alerts
Sprint 3	C2 beacon detection and TLS/QUIC metadata context	Prototype complete	Beacon fixture: 8 events, 1 alert

Production dataset acquisition and deployment-specific calibration remain future work. Smoke-test metrics are never presented as operational accuracy.

3. Environment and the Windows/Linux solution

Zeek is designed for Linux. The development computer runs Windows, so Ubuntu WSL 2 is used as the Linux runtime. Zeek 8.0.10 is installed at `/opt/zeek/bin/zeek`. AegisFlow itself runs on Windows and calls Zeek through a tested WSL bridge.

The bridge performs these operations automatically:

1. Confirm that WSL and Zeek are available. 2. Convert Windows paths such as `C:\project\capture.pcap` to WSL paths such as `/mnt/c/project/capture.pcap`. 3. Run Zeek inside Ubuntu. 4. Write logs back into the Windows project directory. 5. Feed the generated JSON logs to AegisFlow detectors.

The real sample `/home/shukl/zeek-test/sample.pcap` completed this full path. Zeek produced `conn.log`, `dns.log`, `ssl.log`, and capture-filter telemetry.

4. Sprint 0 walkthrough - foundation and reconnaissance

Goal

Prove the smallest complete path from a Zeek connection record to an explainable security alert.

What was built

- Shared `NetworkEvent`, `Evidence`, and `Alert` contracts.
- Zeek `conn.log` JSON reader with line-numbered validation errors.
- Sliding observation windows grouped by source device.
- Vertical port-scan detection: one source probes many ports on one host.
- Horizontal host-scan detection: one source probes the same port across many hosts.
- Cooldown and deduplication to reduce repeated alerts.
- JSON alert output containing confidence, severity, flow identifiers, evidence,

thresholds, and limitations.

Why it matters

This sprint established the contract used by later detectors. The dashboard can eventually display alerts from different attacks without threat-specific UI code.

5. Sprint 1 walkthrough - PCAP, WSL, health, and DDoS

Goal

Accept a raw capture through Zeek and identify high-rate connection behaviour.

What was built

- Windows-to-WSL Zeek execution bridge.
- PCAP-to-Zeek conversion command.
- Reusable keyed sliding-window engine with bounded out-of-order support.
- SYN-flood detection using incomplete connection attempts and their ratio.
- UDP-flood detection using packet and byte volume.
- Replay-health report with event count, event span, ordering, and processing rate.
- Dataset manifest and leakage-safe split policy for CICDDoS2019.

Real-machine result

The real sample capture produced 12 normalized connection events over approximately 120 seconds. No attack alert was raised because the sample did not cross the scan or DDoS thresholds. Zero alerts on ordinary traffic is a valid result.

6. Sprint 2 walkthrough - DNS analytics and first trained model

Goal

Detect two different DNS threats without treating every attack as the same ML task.

DNS-tunnelling path

The detector groups queries by client and approximate base domain inside a sliding window. It measures:

- query count;
- unique subdomain labels;
- average subdomain length;
- average character entropy.

An alert requires high volume and diversity plus long or high-entropy labels. CDN, endpoint-security, hosted telemetry, and allowlist cases are documented because they can resemble tunnelling.

DGA model training

The first model is an inspectable character 3-gram Multinomial Naive Bayes model. For example, `google.com` is converted into overlapping groups such as `^go`, `goo`, `oog`, and `ogl`. Training learns which groups are more frequent in benign or synthetic DGA-like domains.

Training produced three artefacts:

1. A JSON model containing learned n-gram counts and class totals. 2. A JSON evaluation report with confusion-matrix metrics. 3. A model card describing purpose, data, results, risks, and limitations.

The demonstration split contains 20 training examples and 12 test examples. Duplicate domains cannot appear across splits, and malicious families cannot be shared between train and test. The tiny holdout scored 12 of 12 correctly. This only verifies the pipeline; it is not a real-world accuracy claim.

Demonstration result

- Synthetic DNS threat input: 21 DNS events.
- Output: one DGA-like alert and one DNS-tunnelling alert.
- Real sample DNS log: 7 events and zero alerts.
- Encrypted DNS remains invisible unless telemetry is collected before encryption.

7. Sprint 3 walkthrough - C2 beacon and encrypted metadata

Goal

Detect repeated C2-like callbacks without decrypting TLS or QUIC payloads.

Beacon detection logic

Connections are grouped by source, destination, destination port, and protocol. Once enough repeated connections exist, the detector measures:

- mean time between connections;
- timing coefficient of variation, representing jitter;
- total-byte coefficient of variation;
- number of repeated connections in the window.

A beacon candidate requires a plausible callback interval, low timing variation, and stable transfer sizes. A rare fingerprint cannot create an alert by itself.

TLS and QUIC context

The encrypted-session adapter reads metadata such as:

- TLS or QUIC version;
- cipher;
- visible server name;
- application protocol;
- client and server fingerprints when Zeek provides them;
- handshake establishment and resumption state.

A contextual anomaly score can slightly adjust beacon confidence. The alert still requires timing and size evidence. Payloads are never decrypted.

False-positive handling

- Irregular scheduled traffic is tested and does not alert.
- Highly variable transfer sizes do not alert.
- Approved destination IPs can be allowlisted.
- Software updates, monitoring checks, NAT, packet loss, and observation gaps are

listed as limitations in every alert.

Demonstration result

- Synthetic beacon input: 8 connection events and 8 TLS metadata records.
- Output: one periodic-beacon alert.
- Mean interval: 10.02 seconds.
- Interval variation: 0.0147.
- Size variation: 0.0.
- Real sample: 12 connections, 2 TLS records, and zero C2 alerts.

8. What is rule-based and what is machine learning

Threat	Current method	Why
Vertical/horizontal scans	Statistical thresholds	Fan-out behaviour is direct and explainable
SYN/UDP floods	Statistical windows	Rate, volume, and completion evidence are primary signals
DGA-like domains	Trained character n-gram model	Domain character patterns benefit from supervised learning
DNS tunnelling	Behavioural window	Repetition and subdomain diversity matter more than one name
C2 beaconing	Timing and size statistics	Periodicity can be measured without payload access
TLS/QUIC fingerprint	Supporting metadata only	Fingerprints are not reliable proof of malware by themselves

A separate ML model is therefore not trained for every attack. Models are introduced only when labelled data and learned patterns add value beyond transparent rules.

9. Verification summary

The current suite contains 35 automated tests. It covers:

- valid and malformed Zeek connection and DNS records;
- Windows-to-WSL path translation and Zeek invocation;
- scan, SYN-flood, UDP-flood, DGA, DNS-tunnel, and C2 alerts;
- window expiry and out-of-order events;
- benign contacts, hosted-service domains, scheduled traffic, variable transfers,

allowlists, and fingerprint-only suppression;

- dataset leakage checks and model probability bounds.

10. Current limitations

- DGA metrics use a tiny synthetic dataset; production data is not yet acquired.
- The base-domain function uses a two-label approximation until a reviewed public

suffix snapshot is included.

- Current thresholds require environment-specific calibration.
- Encrypted payloads and ordinary DNS-over-HTTPS contents are not inspected.
- NAT can combine multiple devices behind one source address.
- There is no persistent incident database, analyst API, or dashboard yet.
- Threat correlation and calibrated incident scoring begin in Sprint 4.

11. Next sprint

Sprint 4 will add exfiltration behaviour, incident correlation, deterministic scoring, alert deduplication, and analyst-feedback structures. Approved backup traffic will be tested so that high outbound volume alone does not automatically become an exfiltration incident.

Appendix A - Repeatable commands

From the `aegisflow` directory, set `PYTHONPATH=src` for the current shell.

Test everything

```
python -m unittest discover -s tests -v
```

Verify Zeek in WSL

```
python -m aegisflow.cli check-zeek
```

Train the DNS demonstration model

```
python -m aegisflow.cli train-dns `
  --dataset examples/dns_training_demo.csv `
  --model-output output/models/dns_dga_demo.json `
  --metrics-output output/dns_model_metrics.json `
  --model-card-output output/DNS_MODEL_CARD.md
```

Replay DNS threats

```
python -m aegisflow.cli dns-replay `
  --input examples/zeek_dns_threats.jsonl `
  --model output/models/dns_dga_demo.json `
  --output output/sprint2_dns_alerts.jsonl `
  --report-output output/sprint2_dns_report.json
```

Replay C2 beacon traffic with TLS context

```
python -m aegisflow.cli c2-replay `
  --input examples/zeek_conn_beacon.jsonl `
  --encrypted-input examples/zeek_ssl_beacon.jsonl `
  --output output/sprint3_c2_alerts.jsonl `
  --report-output output/sprint3_c2_report.json
```